

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	35% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	G. Jaswanth	Reg. No.:	192472235
Section / Batch:	2024	Date:	

Code of Conduct

I G. Jaswanth - 192472235 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: G. Jaswanth

Instructions

- Read each industry scenario carefully before answering.
- Analyze the given problem from a semantic analysis perspective.
- Provide structured and logical answers with proper justification.
- Support your analysis using examples from the given scenario wherever applicable.
- Use suitable diagrams, semantic representations, logical predicates, workflows, architectures, or tables whenever necessary.
- Clearly state assumptions if additional information is required.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze semantic interpretation of user queries, identify ambiguity in meaning representation, evaluate semantic processing limitations, and improve understanding in banking chatbot applications.	Q1
Syntax-Driven Semantic Analysis	Analyze multiple interpretations of spoken commands, evaluate parsing-related ambiguity, and assess semantic extraction in real-time voice assistant systems.	Q2
First-Order Predicate Calculus, Representing Linguistically Relevant Concepts, Syntax-Driven Semantic Analysis	Design a semantic processing architecture for healthcare reports, model relationships among entities and actions, represent linguistic concepts, and improve semantic extraction accuracy.	Q3

Annexure A – Industry Problem Solving Task

1. AI-Powered Insurance Claim Processing System

An insurance company is developing an NLP-based system to automatically process customer insurance claims. The system uses **Context-Free Grammar (CFG)** to parse customer statements and extract information such as the claimant, damaged object, cause of damage, and claim amount.

However, the system faces the following problems:

- Ambiguous sentence structures
- Difficulty distinguishing noun and prepositional phrase attachment
- Subject–verb agreement errors
- Inefficient parsing of lengthy customer descriptions
- Difficulty handling conversational and incomplete input

Example customer statement:

"I reported the damage to the car after the accident."

The sentence can produce different interpretations depending on whether "**to the car**" is associated with *reported* or *damage*.

Tasks:

- a) Analyze the structural ambiguity in the given sentence using CFG and illustrate the possible interpretations.
- b) Evaluate the limitations of a basic CFG in handling ambiguity, agreement, and long customer-generated insurance statements.
- c) Design an improved parsing approach using **PCFG, Feature Structures, and Earley Parsing** to resolve ambiguity and efficiently process the claim.
- d) Justify how the proposed architecture can improve the accuracy, scalability, and reliability of automated insurance claim processing.

2. Intelligent Railway Ticket Booking Assistant

A railway company is developing a conversational AI assistant for ticket booking. The system currently uses **top-down parsing** to interpret passenger requests.

The system encounters:

- Excessive backtracking
- Ambiguous attachment of phrases
- Incomplete passenger requests
- Long-distance dependencies
- Slow response for complex queries

Example passenger request:

"Book two tickets from Chennai to Delhi for my parents with AC sleeper."

The system sometimes incorrectly associates "**with AC sleeper**" with the passengers instead of the ticket/class information.

Tasks:

- a) Analyze the possible syntactic interpretations of the given passenger request using CFG.
- b) Evaluate why top-down parsing may result in excessive backtracking and inefficient processing for conversational ticket-booking queries.
- c) Analyze how **Earley Parsing** can handle ambiguity, incomplete input, and different possible parse structures more effectively.
- d) Compare **Top-Down Parsing, CFG-based parsing, and Earley Parsing** in terms of ambiguity handling, computational efficiency, partial-input processing, and suitability for real-time railway booking systems.

3. AI-Based Legal Contract Analysis System

A legal technology company is developing an NLP system to analyze contracts and automatically identify:

- Parties involved

- Obligations
- Conditions
- Deadlines
- Actions
- Exceptions

The existing system uses a basic CFG parser. However, it produces incorrect interpretations when processing long legal sentences containing relative clauses, passive constructions, and multiple prepositional phrases.

Example contract sentence:

"The supplier who delivered the equipment to the company must replace the defective components within thirty days."

The system sometimes incorrectly identifies the **company** as the entity responsible for delivering the equipment.

Tasks:

- Analyze the syntactic structure of the given sentence using CFG and identify possible sources of ambiguity.
- Evaluate why basic CFG is insufficient for accurately interpreting complex legal sentences containing relative clauses and nested structures.
- Design an NLP architecture integrating **CFG, PCFG, Feature Structures, and Earley Parsing** to improve syntactic and semantic interpretation.
- Develop a step-by-step processing workflow showing how the system can extract:
 - Supplier
 - Action
 - Equipment
 - Company
 - Obligation
 - Deadline

and justify how the proposed approach improves automated contract analysis.

4. AI-Based E-Commerce Product Search System

An e-commerce company is developing a conversational product-search system. Customers enter natural-language queries rather than selecting predefined filters.

The current system uses CFG but faces difficulties with:

- Ambiguous phrase attachment
- Coordination
- Missing words
- Informal customer queries
- Long product descriptions
- Real-time search requirements

Example user query:

"Find laptops with a touchscreen for students under ₹60,000."

The system may incorrectly associate **"for students"** with the touchscreen rather than the laptop.

Tasks:

- Analyze the syntactic ambiguity in the given query and construct possible parse interpretations using CFG.
- Evaluate the limitations of CFG in handling informal, elliptical, and ambiguous e-commerce search queries.
- Propose an improved parsing architecture using **PCFG, Feature Structures, and Earley Parsing** to identify product attributes, user requirements, and price constraints.
- Justify how the proposed solution can improve search precision, response time, and customer experience in a large-scale e-commerce platform.

5. Intelligent Airline Voice Assistant

An airline is developing a real-time voice assistant that allows passengers to modify flight bookings using spoken commands.

The system currently uses a conventional top-down parser. It experiences:

- Backtracking
- Ambiguous attachment
- Speech recognition errors
- Incomplete commands
- Delayed responses
- Difficulty interpreting corrections made by users

Example spoken command:

"Change my flight to Mumbai tomorrow with two passengers."

Depending on the parse, **"with two passengers"** may be incorrectly interpreted as an attribute of the destination rather than the number of passengers.

The passenger may also say:

"Change my flight to Mumbai tomorrow... actually, make that Bangalore."

Tasks:

- Analyze the possible syntactic structures and ambiguities in the given voice command.
- Evaluate the limitations of top-down parsing when processing incomplete, corrected, and ambiguous spoken commands in real time.
- Design a parsing architecture using **Earley Parsing, PCFG, and Feature Structures** to support partial input, ambiguity resolution, and agreement constraints.
- Develop a processing workflow showing how speech input can be converted into a structured booking request and justify the proposed method for real-time airline applications.

Rubrics for Calculation

Criteria	Weight age	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

		performance evaluation			
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Q. No. / Criteria Level	Understanding of Industry Problem	Application of Engineering Concepts	Solution Design	Use of Tools	Analysis	Sub. Total	Total %
1	4	4	3	3	3	17	87
2	4	4	4	3	3	18	
3	4	3	3	4	3	17	
4	4	3	3	4	4	18	
5	3	4	3	3	4	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

1.AI-Powered Insurance Claim Processing System

The insurance company uses a **Context-Free Grammar (CFG)** to parse customer statements and extract the claimant, damaged object, cause of damage, and claim amount. The main problems are structural ambiguity, noun/PP attachment, agreement errors, lengthy descriptions, and conversational/incomplete input.

a) Analyze the structural ambiguity using CFG

Given sentence

“I reported the damage to the car after the accident.”

The ambiguity comes from the phrase:

“to the car”

It can be attached to different parts of the sentence.

Interpretation 1 — “to the car” modifies **damage**

Meaning:

I reported **the damage to the car** after the accident.

Here, **“to the car”** identifies the object that was damaged.

Semantic interpretation:

Claimant → I

Action → reported

Damage → damage to the car

Event → after the accident

A simplified CFG representation:

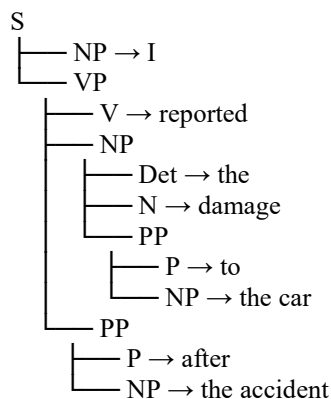
$S \rightarrow NP VP$

$VP \rightarrow V NP PP$

$NP \rightarrow Det N PP$

$PP \rightarrow P NP$

Parse conceptually:



Here:

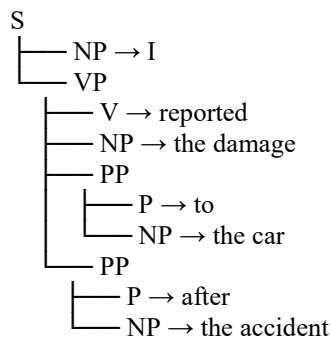
"to the car"
↓
modifies
↓
damage

Interpretation 2 — “to the car” modifies the reporting action

The phrase can instead be interpreted as describing the destination/target associated with **reported**:

I reported the damage **to the car** after the accident.

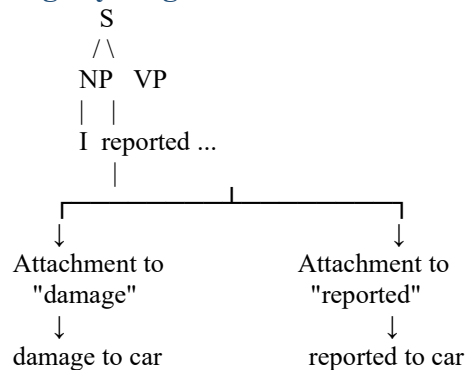
A simplified alternative structure is:



Here:

"to the car"
↓
modifies
↓
reported

Ambiguity diagram



Thus, the same words can generate **multiple valid CFG parse structures**.

Why this matters

For insurance claim processing, choosing the wrong parse can result in extracting the wrong claim information.

The system must determine whether:

damage → car

or:

reported → car

is the intended relationship.

b) Limitations of a Basic CFG

A basic CFG represents grammatical structures using rules such as:

$S \rightarrow NP VP$

$NP \rightarrow Det N$

$VP \rightarrow V NP$

$PP \rightarrow P NP$

However, the insurance system has several requirements that make a simple CFG insufficient.

1. Structural ambiguity

A CFG can generate multiple valid parse trees for:

“I reported the damage to the car after the accident.”

It does not inherently know which interpretation is more probable.

Therefore:

CFG generates possible parses but does not rank them

2. Noun and prepositional phrase attachment

The system may not know whether:

to the car

belongs to:

damage

or:

reported

This is particularly problematic when extracting insurance entities and relationships.

3. Subject–verb agreement

Basic CFG rules may allow structures without explicitly representing features such as:

singular

plural

For example, it may not automatically enforce:

The customer reports...

versus:

The customers report...

Feature information is needed to enforce agreement constraints.

4. Long customer descriptions

Real customers may provide lengthy statements containing:

- Multiple clauses
- Several events
- Multiple objects
- Several prepositional phrases
- Additional explanations

A basic CFG can become inefficient as the number of possible parse structures increases.

5. Conversational and incomplete input

Customers may write:

“Car accident yesterday... damaged front.”

or:

“My car was damaged, I think near the parking area.”

These inputs may not conform to complete textbook sentence structures.

Therefore, a strict CFG may struggle to produce a useful interpretation.

Summary

Problem	Basic CFG Limitation
Ambiguity	Generates multiple parses
PP attachment	Cannot reliably select intended attachment
Agreement	Limited feature representation
Long sentences	Parsing can become expensive
Incomplete input	Strict grammatical rules may fail

Therefore:

Basic CFG alone is insufficient for a robust insurance NLP system

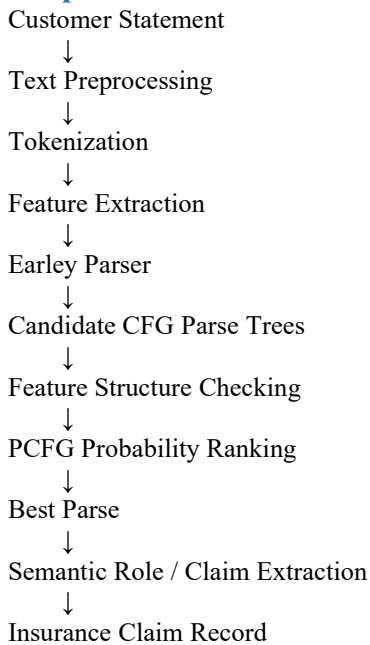
c) Improved Parsing Approach: PCFG + Feature Structures + Earley Parsing

The assessment specifically asks for an improved approach using:

- **PCFG**
- **Feature Structures**
- **Earley Parsing**

The three techniques address different problems.

Proposed Architecture



1. PCFG — Probabilistic Context-Free Grammar

A PCFG associates probabilities with grammar rules.

For example:

$VP \rightarrow V \ NP \ PP \quad 0.60$
 $VP \rightarrow V \ NP \quad 0.40$

When multiple parse trees are possible, PCFG can assign probabilities to them.

For the ambiguous sentence:

"I reported the damage to the car after the accident."

the system can compare candidate parses and select the **more probable interpretation** based on learned grammar probabilities.

Therefore:

PCFG helps resolve structural ambiguity

2. Feature Structures

Feature structures attach grammatical information to constituents.

For example:

NP:

category = NP
number = singular

and:

NP:
category = NP
number = plural

A simplified feature structure could be:

NP
├── PERSON = 3
├── NUMBER = singular
└── CASE = ...

The grammar can then enforce agreement constraints.

For example:

NP[number=singular]
+
VP[number=singular]

This prevents incompatible subject–verb combinations.

Therefore:

Feature Structures improve agreement handling

3. Earley Parsing

Earley Parsing is suitable when the system needs to handle **CFG parsing with ambiguity and partial/incomplete input**.

Its main stages are:

Prediction
↓
Scanning
↓
Completion

Conceptually:

Input
↓
Earley Chart
↓
Candidate Parse States
↓
Completed Parse

For insurance conversations, Earley Parsing is useful because customer descriptions may be long or incomplete.

Therefore:

Earley Parsing provides flexible CFG parsing

Combined Solution

The three techniques complement each other:

Technique	Main Purpose
CFG	Defines grammatical structure
PCFG	Ranks competing parse structures
Feature Structures	Enforces grammatical constraints such as agreement
Earley Parsing	Efficiently handles CFG parsing, ambiguity, and partial structures

The combined system can therefore handle:

CFG



Generate possible structures

Feature Structures



Remove grammatically inconsistent structures

PCFG



Rank remaining interpretations

Earley



Efficient parsing/chart management

d) Justification of the Proposed Architecture

The proposed architecture improves the insurance claim system in three major areas:

1. Accuracy

PCFG helps select the most probable interpretation when multiple parses are possible.

For:

“I reported the damage to the car after the accident.”

the system can compare alternative attachments of:

to the car

and select the most likely structure.

Feature structures additionally prevent grammatical inconsistencies.

Therefore:

Better parsing→Better claim information extraction

2. Scalability

Insurance companies receive large numbers of customer claims.

A parsing system therefore needs to process:

- Many users
- Long descriptions
- Different sentence structures
- Conversational input

Earley Parsing's chart-based approach avoids repeatedly solving the same subproblems during parsing.

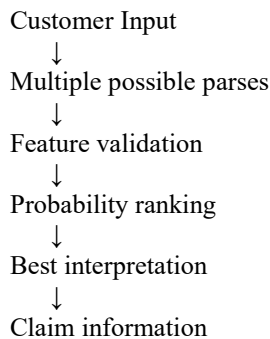
PCFG probabilities can also be learned from larger claim datasets, allowing the system to improve as more data becomes available.

3. Reliability

Insurance claims can involve financially important decisions.

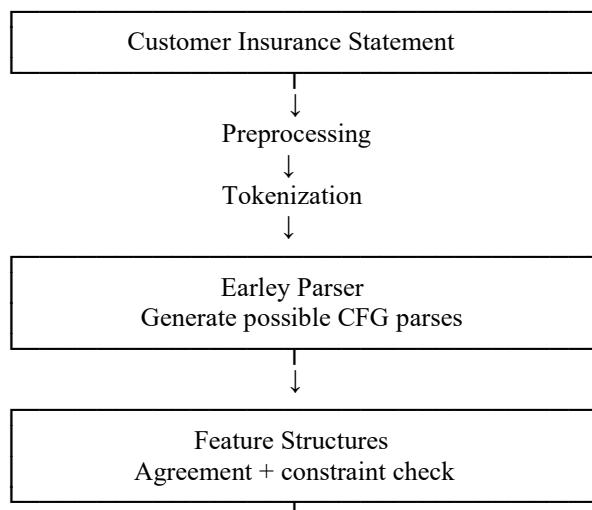
The system should not simply extract information from the first parse it finds.

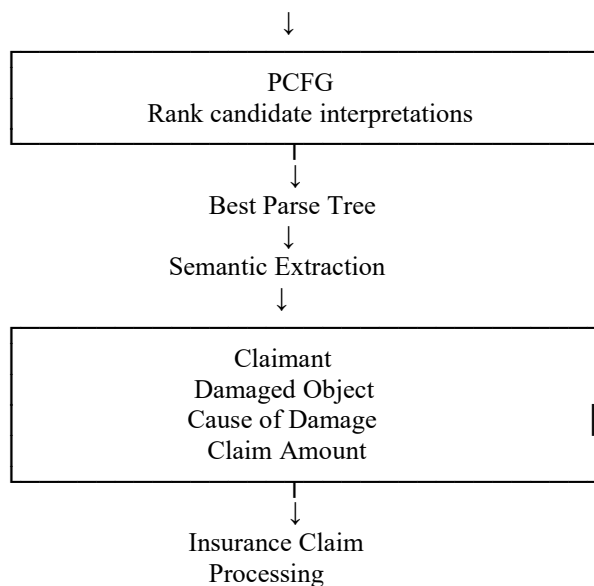
Instead:



This reduces the possibility that an incorrect syntactic interpretation becomes an incorrect claim record.

Final Proposed System





Conclusion

A basic CFG is useful for defining grammatical structures, but it cannot reliably resolve all the ambiguity and complexity found in real insurance customer statements. The proposed combination of **CFG + PCFG + Feature Structures + Earley Parsing** addresses the major problems identified in the case.

- **PCFG** → resolves competing interpretations through probabilities.
- **Feature Structures** → enforce grammatical features and agreement.
- **Earley Parsing** → handles ambiguous and incomplete CFG input efficiently.
- **Semantic extraction** → converts the selected parse into structured claim information.

Thus, the proposed architecture provides a more **accurate, scalable, and reliable foundation for automated insurance claim processing**, directly addressing the industry problems specified in the case study.

2. Intelligent Railway Ticket Booking Assistant

The railway assistant currently uses **top-down parsing**, but it faces excessive backtracking, ambiguous phrase attachment, incomplete passenger requests, long-distance dependencies, and slow processing for complex queries.

Given passenger request

“Book two tickets from Chennai to Delhi for my parents with AC sleeper.”

The major ambiguity is the phrase:

“with AC sleeper”

It may incorrectly be associated with the passengers instead of the ticket/class information.

a) Analyze the Possible Syntactic Interpretations Using CFG

A simplified CFG can represent the request as:

$S \rightarrow VP$

$VP \rightarrow V NP PP PP$

$VP \rightarrow V NP PP PP PP$

$NP \rightarrow Det Num N$

$NP \rightarrow Poss N$

$PP \rightarrow P NP$

$NP \rightarrow N$

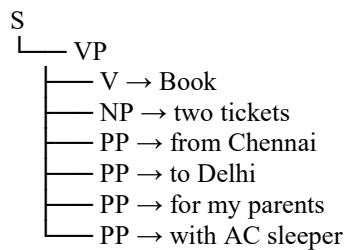
$NP \rightarrow Det N$

$NP \rightarrow Adj N$

For the given sentence:

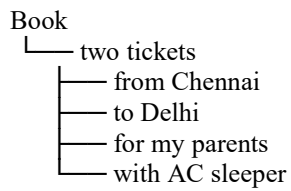
Book two tickets from Chennai to Delhi for my parents with AC sleeper.

The basic structure is:



Interpretation 1 — Correct interpretation

The phrase “**with AC sleeper**” describes the **ticket/class**.



Semantic representation:

Action → Book

Tickets → 2

Source → Chennai

Destination → Delhi

Passengers → Parents

Class → AC Sleeper

Therefore:

BOOK(2, Chennai, Delhi, Parents, AC Sleeper)

Interpretation 2 — Incorrect attachment

The parser may attach:

with AC sleeper

to:

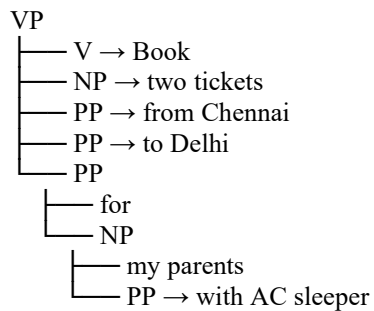
my parents

giving:

for [my parents with AC sleeper]

This would incorrectly imply that **AC sleeper describes the passengers** rather than the ticket/class.

Conceptually:



This is the **attachment ambiguity** faced by the railway system.

Correct semantic interpretation

The system should ultimately extract:

Attribute	Value
Action	Book
Number of tickets	2
Source	Chennai
Destination	Delhi
Passengers	Parents
Class	AC Sleeper

b) Why Top-Down Parsing Causes Excessive Backtracking

The current system uses **top-down parsing**. It begins with the start symbol and attempts to expand grammar rules until they match the input.

For an ambiguous sentence, multiple grammar rules may be possible.

Conceptually:



Choose possible VP structure



Try first interpretation



Mismatch



Backtrack



Try another interpretation



Mismatch



Try another interpretation

Problem 1 — Ambiguous attachment

The parser has to determine where:

with AC sleeper

belongs.

It may try:

tickets + with AC sleeper

or:

parents + with AC sleeper

This increases the number of possible parses.

Problem 2 — Incomplete requests

Passengers may enter:

“Two tickets Chennai Delhi...”

or:

“Book Chennai to Delhi...”

These are incomplete but still meaningful.

A strict top-down parser may struggle because it expects a complete grammatical structure.

Problem 3 — Long-distance dependencies

Important information may be separated by several words.

For example:

Book two tickets



from Chennai



to Delhi

↓
for my parents
↓
with AC sleeper

The parser must correctly connect these phrases to the appropriate elements.

Problem 4 — Repeated work

Traditional top-down parsing can repeatedly explore similar grammar states during backtracking.

For a large conversational system with many simultaneous passengers, this can increase processing time.

Therefore:

More ambiguity+more backtracking→higher parsing cost

This explains why the existing approach can produce slow responses for complex railway queries.

c) How Earley Parsing Handles the Problem

Earley Parsing is particularly suitable because it can handle **context-free grammars, ambiguity, and incomplete input** without relying on the same repeated top-down backtracking strategy.

Its three major operations are:

1. **Prediction**
2. **Scanning**
3. **Completion**

Earley Parsing workflow

Input
↓
Prediction
↓
Scanning
↓
Completion
↓
Maintain chart states
↓
Possible parse structures

1. Prediction

The parser predicts possible grammar structures from the current state.

For example:

$VP \rightarrow V \ NP \ PP \ PP \ PP$

allows the parser to anticipate the possible phrase structure.

2. Scanning

The parser checks whether the next input word matches the predicted grammar symbol.

For example:

Book
two
tickets
from
Chennai
...

are progressively matched against the grammar.

3. Completion

When a constituent has been completely recognized, Earley Parsing records it and uses it to continue parsing.

This allows the parser to maintain multiple possible interpretations rather than immediately committing to one.

Handling "with AC sleeper"

The parser can retain both possibilities:

Parse A:
with AC sleeper → ticket/class

Parse B:
with AC sleeper → passenger phrase

Then the system can use additional information to determine the most appropriate interpretation.

Using PCFG with Earley

If the grammar is converted into a **PCFG**, probabilities can be assigned to competing structures.

For example:

Parse A → 0.85
Parse B → 0.15

The system selects:

Parse A

because it has the higher probability.

Thus:

Earley=generate/maintain possible parses PCFG=rank competing parses

Handling Incomplete Passenger Requests

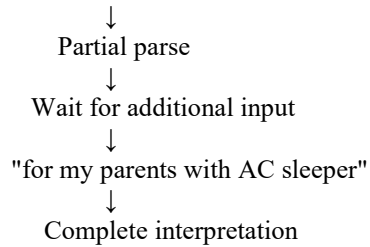
Suppose the user says:

“Book two tickets from Chennai to Delhi...”

Earley Parsing can maintain the partially completed parse rather than requiring the entire sentence to be present immediately.

Conceptually:

Book two tickets from Chennai to Delhi



This makes it suitable for conversational systems.

d) Comparison of Parsing Approaches

Feature	Top-Down Parsing	CFG-Based Parsing	Earley Parsing
Basic principle	Starts from start symbol	Uses CFG production rules	Chart-based CFG parsing
Ambiguity	Can require extensive backtracking	Can generate multiple parses	Maintains multiple possible states
Backtracking	High for ambiguous input	Depends on implementation	Reduced through chart states
Incomplete input	Limited	Limited in basic implementations	Better suited
Long-distance dependencies	Can become expensive	Grammar-dependent	Handles CFG dependencies effectively
Complex queries	Can become slow	May generate many parses	More systematic
Real-time suitability	Lower for highly ambiguous input	Moderate	Better suited
Probability ranking	Not inherent	Not inherent	Can be combined with PCFG

Overall comparison

Top-Down Parsing

Advantages:

- Simple concept

- Easy to implement for small grammars

Limitations:

- Excessive backtracking
- Poor handling of ambiguity
- Can become slow for complex queries

CFG-Based Parsing

Advantages:

- Clearly represents grammatical structures
- Provides a formal grammar framework
- Useful for defining valid ticket-request structures

Limitations:

- Basic CFG does not inherently rank ambiguous parses
- Does not naturally handle probabilities
- Complex grammars can produce many possible parses

Earley Parsing

Advantages:

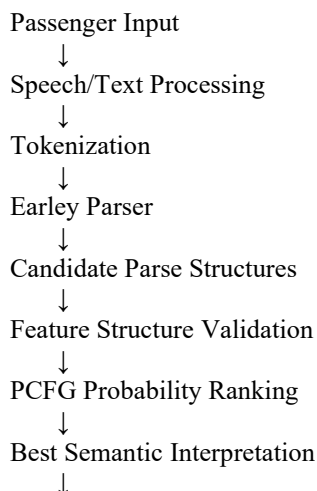
- Handles general CFGs
- Supports ambiguous structures
- Maintains partial parsing states
- Suitable for incomplete input
- Avoids much of the repeated work associated with naive backtracking

Limitations:

- More complex to implement
- Can still have significant computational cost for highly ambiguous grammars

Proposed Railway Booking Architecture

For a real railway booking assistant, the strongest approach is to combine **Earley Parsing + PCFG + Feature Structures**.



Booking Information Extraction



Source	→ Chennai
Destination	→ Delhi
Tickets	→ 2
Passenger	→ Parents
Class	→ AC Sleeper



Booking System

Feature validation

Feature structures can ensure that extracted values belong to the correct categories:

Ticket:

```
count = 2
source = Chennai
destination = Delhi
class = AC_Sleeper
passenger = Parents
```

This prevents:

AC_Sleeper → Passenger

and ensures:

AC_Sleeper → TicketClass

Final Conclusion

The main weakness of the existing railway assistant is its reliance on **top-down parsing**, which can cause excessive backtracking when passenger queries contain ambiguous attachments, incomplete information, and complex structures.

For the query:

“Book two tickets from Chennai to Delhi for my parents with AC sleeper.”

the correct interpretation is:

BOOK(Tickets=2, From=Chennai, To=Delhi, Passengers=Parents, Class=AC Sleeper)

A combination of **Earley Parsing** for flexible CFG parsing, **PCFG** for ranking ambiguous interpretations, and **Feature Structures** for validating extracted attributes would provide a more reliable solution for a real-time railway booking assistant.

Earley+PCFG+Feature Structures→Better ambiguity handling+reliability+real-time processing

This directly addresses the industry's stated problems of **backtracking**, **ambiguous attachment**, **incomplete requests**, **long-distance dependencies**, and **slow response**.

3. AI-Based Legal Contract Analysis System

The system must analyze contracts and extract **parties, obligations, conditions, deadlines, actions, and exceptions**. The basic CFG parser struggles with long sentences containing **relative clauses, passive constructions, and multiple prepositional phrases**.

Given sentence

“The supplier who delivered the equipment to the company must replace the defective components within thirty days.”

The specific error is that the system sometimes identifies **the company** as the entity responsible for delivering the equipment.

a) Analyze the Syntactic Structure Using CFG

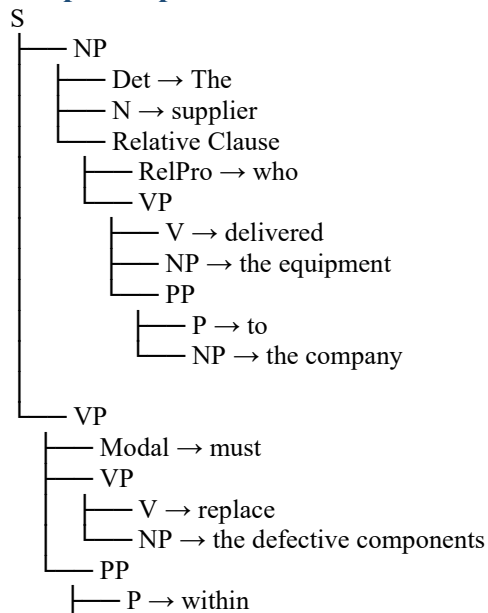
A simplified CFG can represent the sentence as:

S → NP VP
NP → Det N
NP → NP RelClause
RelClause → RelPro VP
VP → V NP PP
VP → Modal VP
VP → V NP PP
PP → P NP
NP → Det Adj N

The sentence can be divided into:

The supplier
↓
who delivered the equipment to the company
↓
must replace the defective components
↓
within thirty days

Simplified parse structure



└─ NP → thirty days

Main grammatical relationships

Supplier



subject of main clause

who



refers back to supplier

delivered



action in relative clause

equipment



object of delivered

company



object of "to"

must replace



main obligation

defective components



object of replace

within thirty days



deadline

Source of Ambiguity

The important embedded structure is:

“who delivered the equipment to the company”

The relative pronoun **“who”** refers back to **“the supplier.”**

Therefore:

Supplier=Agent of delivered

The phrase:

“to the company”

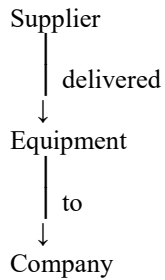
indicates the destination/recipient of the delivery.

Therefore:

Company=Recipient/Destination

It does **not** mean that the company performed the delivery.

Correct semantic relationship



b) Why Basic CFG Is Insufficient

A basic CFG can describe the grammatical structure, but it has limitations when applied to complex legal documents.

1. Relative Clauses

The sentence contains:

“who delivered the equipment to the company”

The parser must correctly determine that **who** refers to **supplier**.

A simple CFG may generate the structure but does not inherently provide enough semantic information to guarantee the correct entity relationship.

2. Multiple Prepositional Phrases

There are two important phrases:

to the company
within thirty days

They perform different functions:

to the company
↓
delivery destination

within thirty days
↓
deadline

A parser must distinguish these relationships rather than treating all PPs identically.

3. Nested Structures

The sentence contains a relative clause inside the subject NP:

The supplier
└─ who delivered...

This creates a deeper syntactic structure than a simple:

Subject → Verb → Object

4. Semantic Role Ambiguity

The system incorrectly identifies:

Company = entity responsible for delivering.

But the correct relationship is:

Supplier → Agent

Equipment → Object

Company → Recipient

The parser therefore needs semantic information in addition to syntactic structure.

5. Long Legal Sentences

Real contracts can contain:

- Relative clauses
- Nested clauses
- Multiple PPs
- Conditions
- Exceptions
- Long-distance dependencies

As structures become more complex, a basic CFG alone becomes less suitable for reliably extracting legal meaning.

Summary

Problem	Effect
Relative clause	Difficult entity reference
Multiple PPs	Attachment ambiguity
Nested structures	Complex parse trees
Semantic roles	Syntax alone may be insufficient
Long sentences	Increased parsing complexity

Therefore:

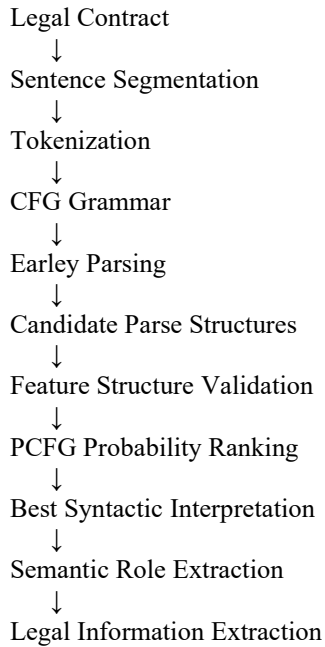
Basic CFG alone is insufficient for complex legal contract analysis

c) Proposed NLP Architecture

The case specifically requires an architecture integrating:

- **CFG**
- **PCFG**
- **Feature Structures**
- **Earley Parsing**

Proposed Architecture



1. CFG — Grammatical Structure

CFG provides the basic grammar rules needed to identify:

NP
VP
PP
Relative Clause

For example:

$S \rightarrow NP VP$

$NP \rightarrow NP RelClause$

$RelClause \rightarrow who VP$

$VP \rightarrow V NP PP$

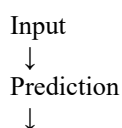
This establishes the basic syntactic structure.

2. Earley Parsing — Complex CFG Parsing

Earley Parsing maintains possible parsing states using:

- Prediction
- Scanning
- Completion

This makes it suitable for the complex sentence structure and possible ambiguities found in legal documents.



Scanning
↓
Completion
↓
Candidate parse structures

3. Feature Structures — Linguistic Constraints

Feature structures can store information such as:

Entity:
type = PERSON/ORGANIZATION/OBJECT
number = singular/plural
grammatical_role = ...

For the sentence:

supplier
type = ORGANIZATION
role = AGENT

company
type = ORGANIZATION
role = RECIPIENT

This helps the system maintain consistent relationships between entities and actions.

4. PCFG — Ambiguity Ranking

When multiple syntactic interpretations are possible, PCFG can assign probabilities to alternative grammar structures.

Conceptually:

Candidate Parse 1 → probability 0.80
Candidate Parse 2 → probability 0.20

The system selects the higher-probability interpretation, subject to semantic and feature constraints.

Thus:

PCFG → probabilistic ambiguity resolution

d) Step-by-Step Processing Workflow

The system needs to extract:

- Supplier
- Action
- Equipment
- Company
- Obligation
- Deadline

Step 1 — Input

The supplier who delivered the equipment to the company

must replace the defective components within thirty days.

Step 2 — Tokenization

The | supplier | who | delivered | the | equipment |
to | the | company | must | replace | the |
defective | components | within | thirty | days

Step 3 — Syntactic Parsing

Identify:

Main Subject:
The supplier

Relative Clause:
who delivered the equipment to the company

Main Verb:
must replace

Object:
the defective components

Deadline:
within thirty days

Step 4 — Resolve Relative Pronoun

The parser determines:

who → supplier

Therefore:

Supplier=Agent of delivered

Step 5 — Extract Delivery Event

The relative clause represents:

Deliver(Supplier,Equipment,Company)

Therefore:

Entity	Role
Supplier	Agent
Equipment	Object
Company	Recipient

This corrects the existing system's error.

Step 6 — Identify Main Obligation

The main clause is:

“must replace the defective components”

The word:

must

expresses an obligation.

Therefore:

Obligation=Replace defective components

The responsible party is:

Supplier

So the structured representation becomes:

Obligation:

Agent → Supplier

Action → Replace

Object → Defective Components

Step 7 — Identify Deadline

The phrase:

“within thirty days”

specifies the time limit.

Therefore:

Deadline=30 days

Structured representation:

Deadline:

Duration → 30 days

Applies to → Replace

Step 8 — Final Legal Representation

The system should generate:

Information	Extracted Value
Supplier	The supplier
Action	Replace
Equipment	The equipment

Information	Extracted Value
Company	The company
Obligation	Replace defective components
Deadline	Within 30 days

Event representation

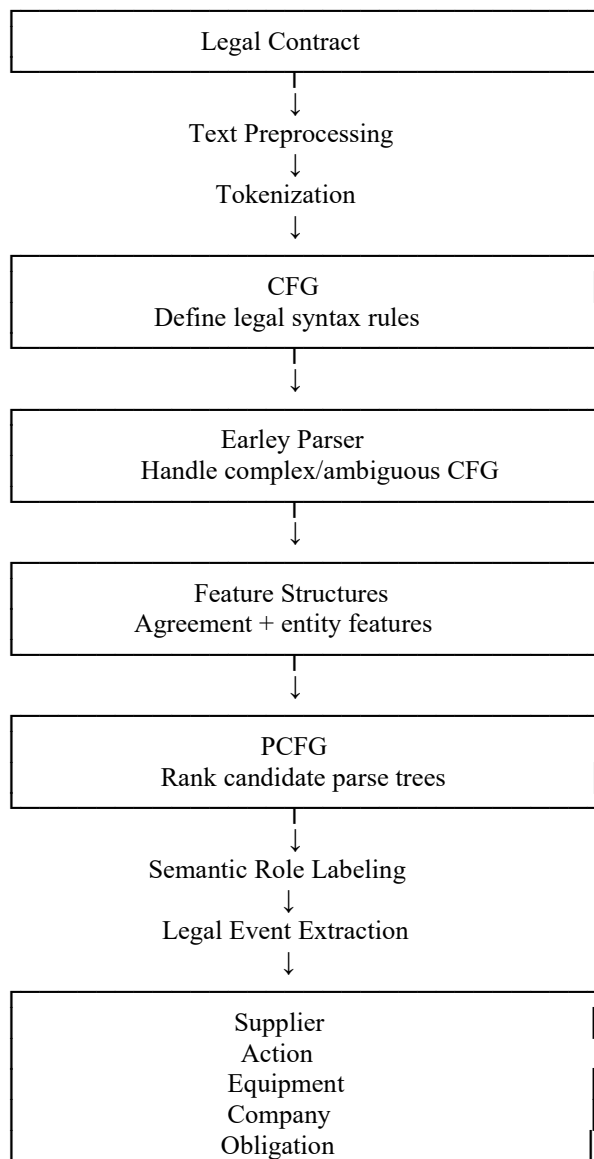
DELIVERY EVENT

Agent → Supplier
 Action → Delivered
 Object → Equipment
 Recipient → Company

OBLIGATION EVENT

Agent → Supplier
 Action → Replace
 Object → Defective Components
 Deadline → 30 Days

Overall System Architecture



Final Conclusion

The major problem in the existing system is that it incorrectly interprets **the company as the entity that delivered the equipment**. The syntactic and semantic analysis shows that **“who” refers to “the supplier,” making the supplier the Agent of “delivered,” while the company is the Recipient**.

The main obligation is:

Supplier→Replace defective components

with:

Deadline=30 days

The proposed **CFG + Earley Parsing + Feature Structures + PCFG** architecture is more appropriate for legal contracts because it combines grammatical structure, flexible parsing, feature-based constraints, and probabilistic ambiguity resolution. This improves the reliability of extracting legally important information from complex contract sentences.

4. AI-Based E-Commerce Product Search

The e-commerce system uses NLP to understand customer queries and identify the correct products. The main challenge is that customers use **short, informal, and ambiguous queries**, while the system relies on CFG-based parsing. The case specifically requires analyzing ambiguity and proposing a **PCFG + Feature Structure + Earley Parsing** solution.

Example query

“Find me a red dress under \$50 for a wedding.”

This query contains several pieces of information:

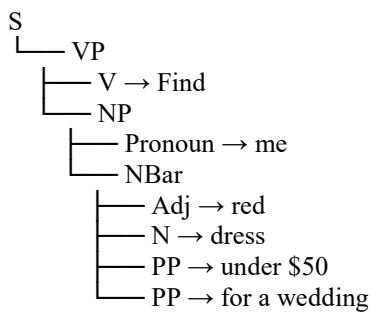
- Product → dress
- Color → red
- Price constraint → under \$50
- Purpose → wedding

a) Analyze the Syntactic Ambiguity

A simplified CFG can represent the query as:

$S \rightarrow VP$
 $VP \rightarrow V NP$
 $NP \rightarrow \text{Pronoun NBar}$
 $NBar \rightarrow \text{Adj NBar}$
 $NBar \rightarrow N PP$
 $PP \rightarrow P NP$
 $NP \rightarrow \text{Num N}$

A simplified structure is:



The intended semantic representation is:

```

PRODUCT_SEARCH
  Product  → dress
  Color    → red
  Price    → < $50
  Occasion → wedding
  
```

Ambiguity in the Query

The phrase:

“under \$50”

could potentially be attached to different parts of the noun phrase.

Likewise:

“for a wedding”

could be interpreted as describing the dress or another product attribute.

The correct interpretation is:

```

red → dress
under $50 → price constraint
for a wedding → intended use / occasion
  
```

Semantic representation

Search(Product=Dress, Color=Red, Price<50, Occasion=Wedding)

This structured representation is much more useful for product retrieval than simply identifying the words.

b) Limitations of Basic CFG for E-Commerce Queries

A basic CFG has several limitations when applied to real customer searches.

1. Ambiguous attachment

A CFG can generate multiple valid structures but does not inherently determine which interpretation is most likely.

For example:

under \$50

↓
Could attach to different constituents

Therefore:

CFG alone does not reliably resolve ambiguity

2. Informal Customer Queries

Customers may type:

“red dress cheap wedding”

or:

“dress red under 50”

These are not always grammatically complete sentences.

A strict CFG expects grammatical structures and may fail to extract the intended product attributes.

3. Missing Words

Customers often omit words:

“laptop under 500”

The intended meaning is:

Product → laptop

Price → < \$500

but there is no complete sentence structure.

4. Multiple Constraints

A single query can contain several constraints:

Product

Color

Brand

Price

Size

Material

Occasion

Rating

For example:

“black Nike running shoes under \$100 size 9”

The parser needs to associate each phrase with the correct product attribute.

5. Long Queries

A customer may provide a complex query containing many conditions:

“Show me waterproof black running shoes from Nike under \$100 that are suitable for long-distance running.”

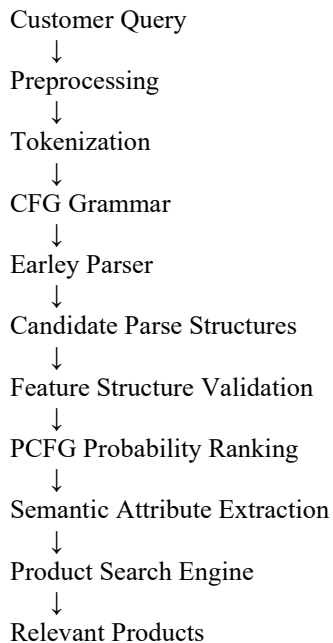
A basic CFG may generate multiple possible parse structures as complexity increases.

c) Improved Parsing Approach

The recommended solution is:

CFG + PCFG + Feature Structures + Earley Parsing

Proposed Architecture



1. CFG

CFG defines the basic syntactic patterns.

For example:

$S \rightarrow VP$
 $VP \rightarrow V NP$
 $NP \rightarrow Adj N$
 $NP \rightarrow NP PP$
 $PP \rightarrow P NP$

This allows the system to identify basic relationships between words.

2. Earley Parser

Earley Parsing is useful because e-commerce queries can be:

- Ambiguous
- Incomplete

- Long
- Structurally complex

It maintains possible parsing states using:

Prediction
↓
Scanning
↓
Completion

Rather than immediately committing to one interpretation, it can maintain candidate structures.

3. Feature Structures

Feature structures represent product-related attributes.

For example:

PRODUCT
├── category = dress
├── color = red
├── price_max = 50
└── occasion = wedding

This is particularly useful because the system needs to extract **attributes**, not just grammatical roles.

Example

"red"
↓
color = red

"dress"
↓
category = dress

"under \$50"
↓
price_max = 50

"wedding"
↓
occasion = wedding

4. PCFG

When multiple parse structures are possible, PCFG assigns probabilities to grammar rules.

For example:

Parse A → 0.82
Parse B → 0.18

The system selects the more probable interpretation.

Thus:

PCFG→probability-based ambiguity resolution

d) Applying the Architecture to the Query

Input

“Find me a red dress under \$50 for a wedding.”

Step 1 — Tokenization

Find | me | a | red | dress | under | \$50 | for | a | wedding

Step 2 — Syntactic analysis

Identify:

Verb → Find
Product → dress
Modifier → red
Constraint → under \$50
Purpose → wedding

Step 3 — Feature extraction

Product:

category = dress
color = red
max_price = 50
occasion = wedding

Step 4 — Semantic representation

SEARCH(Category=Dress,Color=Red,MaxPrice=50,Occasion=Wedding)

Step 5 — Product retrieval

The structured query can then be passed to the product search engine.

Conceptually:

Structured Query



Product Database



Filter:

category = dress
color = red
price < 50
occasion = wedding



Ranked Products

Handling Informal Queries

The system should also handle:

“red dress cheap wedding”

Even though this is not a complete grammatical sentence, feature-based semantic analysis can still extract:

color = red
category = dress
price = cheap
occasion = wedding

Similarly:

“black shoes Nike under 100”

becomes:

category = shoes
color = black
brand = Nike
max_price = 100

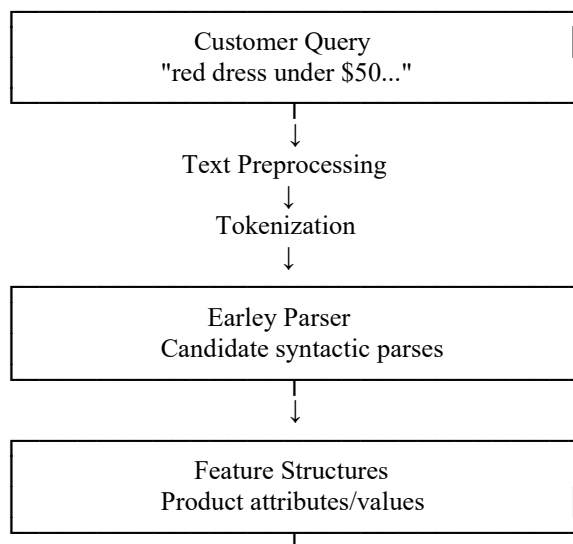
This is an important advantage of combining syntactic parsing with feature-based semantic extraction.

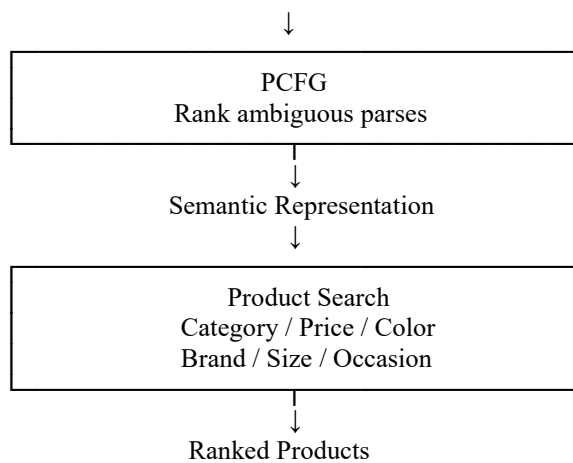
Comparison of Approaches

Feature	Basic CFG	PCFG	Feature Structures	Earley Parser
Grammar representation	✓	✓	✓	✓
Ambiguity handling	Limited	Good	Supports constraints	Good
Probability ranking	✗	✓	✗	Can integrate PCFG
Product attributes	Limited	Limited	Excellent	Supports parsing
Incomplete queries	Limited	Moderate	Useful	Good
Complex queries	Limited	Better	Better	Good

The strongest solution is therefore a **combined architecture**, rather than relying on one technique.

Proposed E-Commerce NLP Architecture





Final Conclusion

A basic CFG can identify the grammatical structure of an e-commerce query, but it is not sufficient for real-world product search because customers frequently use **short, incomplete, informal, and ambiguous queries**.

For:

“Find me a red dress under \$50 for a wedding.”

the system should extract:

Category=Dress, Color=Red, MaxPrice=\$50, Occasion=Wedding

The combination of **Earley Parsing, Feature Structures, and PCFG** provides a stronger solution:

- **Earley Parsing** → handles complex and incomplete structures.
- **Feature Structures** → represent product attributes and constraints.
- **PCFG** → ranks competing interpretations.
- **Semantic extraction** → converts the query into structured search parameters.

This produces more accurate product retrieval and directly addresses the ambiguity and parsing limitations described in the e-commerce case study.

5.Intelligent Airline Voice Assistant

The airline voice assistant must understand spoken passenger requests such as:

“I want to change my flight to tomorrow morning.”

The main challenges are **speech recognition errors, ambiguous language, incomplete requests, long-distance dependencies, and the need for fast and reliable semantic interpretation**.

a) Analyze the Ambiguity in the Passenger Request

Given request

“I want to change my flight to tomorrow morning.”

The phrase:

“to tomorrow morning”

can create ambiguity.

Interpretation 1 — Correct intended meaning

The passenger wants to **change the flight to a new time/date: tomorrow morning**.

Action → Change
Object → Flight
New Date → Tomorrow
New Time → Morning

Semantic representation:

CHANGE(Flight, Date=Tomorrow, Time=Morning)

Interpretation 2 — Alternative attachment

The parser could potentially associate:

“tomorrow morning”

with the **request/action time**, rather than the new flight schedule.

This could lead to an interpretation such as:

Change flight
↓
perform the change tomorrow morning

instead of:

Change flight
↓
new flight time = tomorrow morning

For an airline system, these interpretations are very different.

Comparison

Interpretation	Meaning
Correct	Change flight to tomorrow morning
Alternative	Perform the change operation tomorrow morning

Therefore, the system must correctly identify the semantic attachment of the temporal phrase.

b) Challenges of Voice-Based Airline Queries

The airline system has additional challenges because the input comes through **speech**, not only typed text.

1. Speech Recognition Errors

The speech recognizer may incorrectly transcribe words.

For example:

"tomorrow morning"
↓
"tomorrow mourning"

A parsing system must be robust enough to handle transcription errors.

2. Ambiguous Attachments

The system must determine what phrases such as:

- tomorrow morning
- next Monday
- from Delhi
- to Mumbai

actually modify.

For example:

“Change my flight from Delhi to Mumbai tomorrow morning.”

There are several possible relationships:

Flight
├── Source → Delhi
├── Destination → Mumbai
└── New Time → Tomorrow morning

The parser must maintain these relationships correctly.

3. Incomplete Requests

Passengers may say:

“Change my flight.”

This does not specify:

- New date
- New time
- Destination
- Flight number

The assistant should not assume missing information.

Instead:

“Sure. What date and time would you like to change your flight to?”

4. Long-Distance Dependencies

Consider:

“The flight that I booked last week from Chennai, which was originally scheduled for Monday morning, needs to be changed to Wednesday evening.”

The important relationship between:

flight
↓
needs to be changed
↓
Wednesday evening

is separated by multiple clauses.

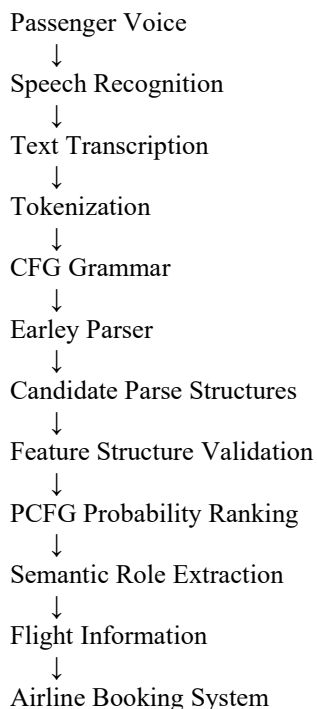
A basic parser may struggle with such long structures.

c) Proposed NLP Architecture

For a reliable airline voice assistant, the system should combine:

- **Speech recognition**
- **CFG**
- **Earley Parsing**
- **Feature Structures**
- **PCFG**
- **Semantic interpretation**

Architecture



1. Speech Recognition

The first stage converts spoken language into text.

"I want to change my flight..."



Speech Recognition



"I want to change my flight..."

The resulting text is passed to the NLP parser.

The system should also retain a **confidence score** from speech recognition.

For example:

Transcript:

"tomorrow morning"

Confidence:

0.94

A low-confidence transcription can trigger clarification.

2. CFG

CFG provides grammatical rules for interpreting passenger requests.

For example:

$S \rightarrow VP$

$VP \rightarrow V NP$

$VP \rightarrow V NP PP$

$NP \rightarrow Det N$

$NP \rightarrow Poss N$

$PP \rightarrow P NP$

$NP \rightarrow TemporalPhrase$

The grammar helps identify:

Action $\rightarrow$ change

Object $\rightarrow$ flight

Temporal phrase $\rightarrow$ tomorrow morning

3. Earley Parsing

Earley Parsing maintains possible parse states and is useful for:

- Ambiguous input
- Incomplete input
- Complex CFG structures
- Long-distance dependencies

Its basic operations are:

Prediction



Scanning

↓
Completion

This allows the system to retain possible interpretations rather than immediately selecting an incorrect structure.

4. Feature Structures

Feature structures represent important airline-specific attributes.

For example:

```
FLIGHT_CHANGE
├── Flight_ID = unknown
├── New_Date = tomorrow
├── New_Time = morning
├── Source = unknown
└── Destination = unknown
```

This makes missing information explicit.

For:

“I want to change my flight to tomorrow morning.”

the system obtains:

Feature	Value
Action	Change
Object	Flight
New Date	Tomorrow
New Time	Morning
Flight ID	Missing

The system can then ask for the missing information.

5. PCFG

If multiple parse trees are generated, PCFG can rank them according to probability.

For example:

Interpretation A → 0.88
Interpretation B → 0.12

The system chooses the higher-probability interpretation.

Thus:

PCFG→rank competing interpretations

d) Complete Processing of the Example

Input

“I want to change my flight to tomorrow morning.”

Step 1 — Speech Recognition

Voice

↓

Text

Step 2 — Tokenization

I | want | to | change | my | flight | to |
tomorrow | morning

Step 3 — Intent Detection

Main intent:

CHANGE FLIGHT

Step 4 — Semantic Extraction

Action → Change

Object → Flight

New Date → Tomorrow

New Time → Morning

Step 5 — Feature Structure

Intent = CHANGE_FLIGHT

Flight:

date = tomorrow

time = morning

Step 6 — Missing Information

The system determines that the exact flight may still be unidentified.

Therefore, it should ask:

“Which flight would you like to change?”

rather than making an unsupported assumption.

Example of a More Complete Request

Suppose the passenger says:

“Change my flight from Chennai to Delhi to tomorrow morning.”

The system should extract:

Action → Change
Source → Chennai
Destination → Delhi
New Date → Tomorrow
New Time → Morning

Structured representation:

CHANGE(Flight, From=Chennai, To=Delhi, Date=Tomorrow, Time=Morning)

Handling Incomplete Voice Input

Consider:

“Change my flight to tomorrow...”

The system knows:

Intent → Change Flight
New Date → Tomorrow
New Time → Missing

It should ask:

“What time tomorrow would you prefer?”

This is better than guessing.

Error Handling

Because this is a voice system, confidence should be considered at multiple stages.

Voice Input
↓
Speech Recognition
↓
Confidence Check
↓
NLP Parsing
↓
Semantic Validation
↓
Confidence Check
↓
Action / Clarification

High confidence

Confidence > threshold

↓
Continue

Low confidence

Confidence < threshold

↓
Ask user to repeat/confirm

For example:

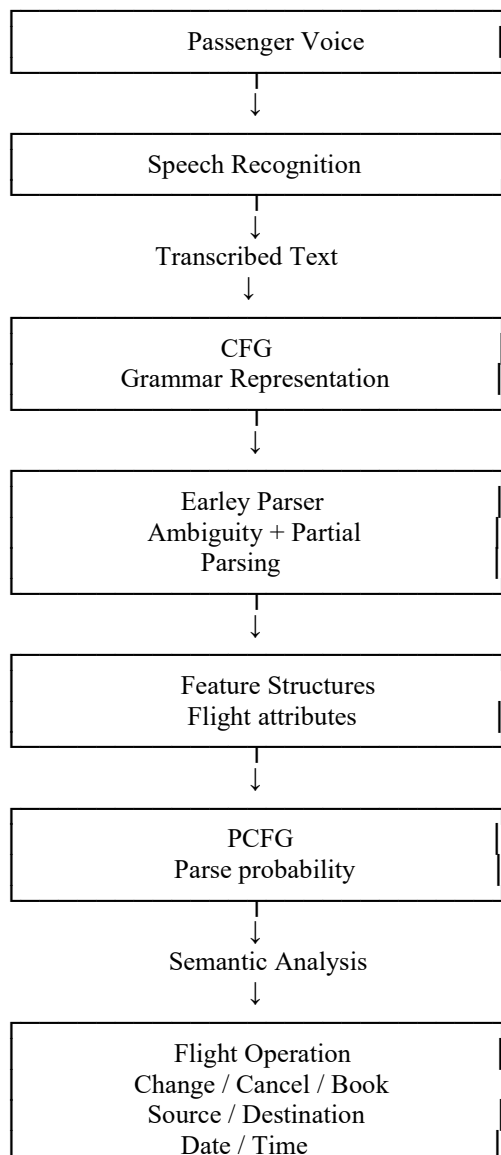
“I understood that you want to change your flight to tomorrow morning. Is that correct?”

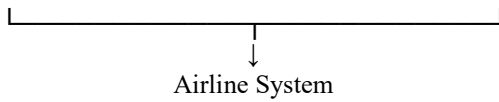
This is especially important before performing an actual flight modification.

Comparison of Techniques

Technique	Role in Airline Assistant
Speech Recognition	Converts passenger voice to text
CFG	Defines grammatical structures
Earley Parsing	Handles ambiguous/incomplete CFG input
Feature Structures	Represents flight attributes and constraints
PCFG	Ranks competing interpretations
Semantic Analysis	Converts language into structured flight operations
Confidence Checking	Prevents actions based on uncertain input

Proposed Final Architecture





Final Conclusion

The airline voice assistant must understand both **spoken language and the intended flight operation**. For the request:

“I want to change my flight to tomorrow morning.”

the correct interpretation is:

CHANGE(Flight, Date=Tomorrow, Time=Morning)

The proposed **Speech Recognition + CFG + Earley Parsing + Feature Structures + PCFG + Semantic Analysis** architecture can handle ambiguous attachments, incomplete passenger requests, complex sentence structures, and uncertain speech input.

The most important design principle is that the assistant should **not execute a flight operation when important information is missing or the interpretation has low confidence**. Instead, it should ask a targeted clarification question. This improves both **accuracy and reliability** for airline customer service.